

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030437

Name : Bandla Venkata Rama Vinay

SEC-8A

PRACTICAL-3:

Aim: Develop a C program to demonstrate process creation using **fork()** and display the process ID and parent process ID using **getpid()** and **getppid()**. Demonstrate parent-child process execution and synchronization using **wait()**.

Observe the execution of the parent and child processes and identify the process relationships and termination sequence.

Procedure :

1. Create the C source file **Practical-3.c** containing the process creation program.
2. Compile the program using GCC and generate the executable **Practical_3**.
3. Execute the program using **./Practical_3**.
4. Observe the PID and PPID values printed before and after **fork()**.
5. Observe the parent waiting for the child and the subsequent termination messages.

Output :

```
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing$ ./Practical_3
Before fork():
PID = 1302
PPID = 334

PARENT PROCESS
PID = 1302
PPID = 334
Child PID = 1303
Parent is waiting for child...
CHILD PROCESS
PID = 1303
PPID = 1302
Child is running...
Child is terminating...
Child terminated.
Parent is terminating...
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing$ |
```

The submitted terminal output shows the original process information before **fork()**, followed by the parent and child process information. The parent waits for the child, the child runs and terminates, and the parent then terminates.

Code:

```
Practical-3.c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5  #include <stdlib.h>
6
7  int main()
8  {
9      pid_t pid;
10
11      printf("Before fork():\n");
12      printf("PID = %d\n", getpid());
13      printf("PPID = %d\n\n", getppid());
14
15      pid = fork();
16
17      if (pid < 0)
18      {
19          printf("Fork failed!\n");
20          return 1;
21      }
22
23      if (pid == 0)
24      {
25          printf("CHILD PROCESS\n");
26          printf("PID = %d\n", getpid());
27          printf("PPID = %d\n", getppid());
28          printf("Child is running...\n");
29
30          sleep(20);
31
32          printf("Child is terminating...\n");
33          exit(0);
34      }
35      else
36      {
```

Code (continued):

```
37     printf("\nPARENT PROCESS\n");
38     printf("PID = %d\n", getpid());
39     printf("PPID = %d\n", getppid());
40     printf("Child PID = %d\n", pid);
41
42     printf("Parent is waiting for child...\n");
43
44     wait(NULL);
45
46     printf("Child terminated.\n");
47     printf("Parent is terminating...\n");
48 }
49
50 return 0;
51 }
```

System Calls

System Call / Function	What it does
fork()	Creates a new child process from the calling parent process.
getpid()	Returns the process ID of the calling process.
getppid()	Returns the process ID of the calling process's parent.
wait()	Makes the parent wait until the child process terminates.
sleep()	Pauses the child process for the specified number of seconds.
exit()	Terminates the child process.

Kernel services involved

1. Process Management

fork() creates the child process, while **wait()** provides synchronization between the parent and child. **exit()** terminates the child process.

2. Process Identification

getpid() and **getppid()** obtain the process ID and parent process ID, allowing the process relationship to be observed.

3. Process Scheduling and Synchronization

The child executes independently after **fork()**. The parent invokes **wait(NULL)** and therefore waits for the child to finish before continuing.

Process flow

Parent process → **fork()** → Parent + Child processes → Child terminates → Parent continues and terminates

Observation :

- Before **fork()**, the program displays the original process PID and PPID.
- **fork()** creates a child process while the parent process continues.
- The parent process displays its PID, PPID, and the newly created child PID.
- The parent waits for the child using **wait(NULL)**.
- The child displays its own PID and the parent's PID.
- The child runs for the specified delay, prints its termination message, and exits.
- After the child terminates, the parent prints its termination message.

Result :

The C program successfully demonstrated process creation using **fork()**, process identification using **getpid()** and **getppid()**, and parent-child synchronization using **wait()**. The execution output clearly shows the parent and child process relationship and their termination sequence.

Practical-3 Completed Successfully